

TutorConnect: Milestone 2

Author: Ali Halmamat

Course: CMPE 172 – Booking and Concurrency

Student ID: 017505410

Code walkthrough:

<https://drive.google.com/file/d/1gKIBK-QHXuc6YweKB-FyWoZ8EiJ4YlaS/view?usp=sharing>

Repository: <https://github.com/mr-uyghur/tutorconnect-cmpe172-milestone1>

Milestone 2 tag: Milestone #2

Where the project is now

At the end of Milestone 1, the app could show tutors and services and such, but nobody could book a session yet. Milestone 2 adds that missing piece.

The student and tutor pages now sit behind a login, which means the user has to login first to get to that page.

Once signed in, a student can now filter open slots by tutor, subject, or date. Also, they can book a slot, see their upcoming appointments and past history, and cancel an appointment that hasn't started yet. A tutor can add a slot, remove one that nobody has booked, and see who has booked with them as well. Pages are still rendered with Thymeleaf, and the data access is still using JDBC with SQL I wrote.

From the booking page to the database

Now when a student clicks Book on a slot, the form posts to `/slots/{id}/book`. It only sends the slot ID, since the slot is already linked to a service in the database at this point.

Spring Security is used for checking two things first, since the session belongs to a CUSTOMER, and the CSRF token is valid.

After that, WebController takes the logged-in user's ID and the slot ID and passes them to BookingService.

Following that, BookingTransaction is called for each try using our BookingService. That class reads the time slot to make sure it hasn't started and doesn't already have a booking. Next, it adds a BOOKED appointment and bumps the version of the slot. These queries are executed by the repositories using JdbcTemplate. The controller reroutes to a confirmation page if the transaction is successful. Refreshing the website does not resubmit the booking form due to the redirect.

Here's the version update: `UPDATE availability_slots SET version = version + 1 WHERE slot_id = ? AND version = ?;`

If this update affects 0 rows, another request modified the slot after we read it. In that situation, the attempt fails. BookingService can retry in a new transaction up to three times, and it can also retry if SQLite throws a temporary busy-lock error. Every retry reads the current state again. If someone else's reservation is now available, the student receives a 409 Conflict and is able to select an alternative slot.

The Java check in BookingTransaction uses the update result directly: `if (slots.bumpVersion(slotId, slot.version()) == 0) throw new VersionConflictException();`

Two students, one slot

Let's say Maya and Jordan are both looking at the same open Java session and hit Book at almost the same moment. In this example, if the only check ran when the page loaded, both of them would think they got it. In that case our version update catches this, since whoever read the slot second is now gonna be working with the stale data.

There's also a second safeguard in the database that doesn't depend on the Java code: `CREATE UNIQUE INDEX uq_active_booking ON appointments(slot_id) WHERE status = 'BOOKED';`

This means that a slot can never have more than one current booking. Canceled appointments are saved in the history, but they do not count against the index. The version check is how the application detects races. The index protects the data even if a future code modification unintentionally removes that check.

The booking method is annotated with `@Transactional(isolation = Isolation.SERIALIZABLE)`. Here's how that maps to ACID:

- **Atomicity:** the version update and the appointment insert either both commit or both roll back.
- **Consistency:** foreign keys, allowed status values, and the unique index keep the stored data valid.
- **Isolation:** SQLite serializes writes, and the version check catches any value that changed after we read it.
- **Durability:** once a booking commits, it's saved in the SQLite file and survives a restart.

The retry loop sits outside the transactional method on purpose. That way each retry gets a brand-new transaction instead of reusing one that already failed.

Login, roles, and cancellation

The login form requires an email address and password. Passwords are saved as BCrypt hashes and JdbcUserDetails for encryption. The service obtains the user's role from the users table. Spring Security saves the session on the server.

CUSTOMER channels include booking and student appointments. PROVIDER routes address tutor availability and appointments. A signed-in user attempting to open a route for the other role receives a 403.

Since checking the route is extremely insufficient to prevent cancellations for our users. The SQL UPDATE matches both the appointment ID and the current customer ID. It also demands that the appointment be BOOKED and that the slot start in the future.

So let's say a student cannot cancel someone else's booking simply by altering the ID in the URL. A bad input returns a 400, a missing record returns a 404, and a slot conflict returns a 409. The user never sees the stack trace.

Moving the database to SQLite

The earlier version used an in-memory H2 database, but the course requires MySQL, PostgreSQL, or SQLite, so I naturally switched to SQLite to keep the database locally for now. This meant replacing H2's date functions and the generated-column trick I had used for the booking rule. The active-booking rule is now the partial unique index shown above. SQLite also reports busy locks and constraint failures differently from H2, so the app translates those errors into booking conflicts.

While I was testing restarts, I found a bug: a sample slot that a tutor had deleted came back after a restart, because the seed script ran again. Now the initializer creates the schema when it's missing but only inserts demo data into a brand-new database file. I tested it again, and the deleted slot stayed deleted after the restart.

Tests

At commit 129f17d on GitHub, mvn clean verify passes on JDK 21.0.10: 55 tests, with no failures or errors. The suite includes:

- service tests for booking and cancellation
- web tests for login and roles

- two race tests

One race test fires two booking calls at once, and the other fires ten. Both check that exactly one call succeeds and that the database ends up with exactly one active booking. Since a race test passing once doesn't prove much, I ran them eight times in a row, and they passed every time. I also did a manual restart check against the file-based SQLite database.

The walkthrough video link and the Milestone 2 tag still need to be added.

Code assistant disclosure

Claude Code helped write the initial Milestone 2 code. Codex helped move the project to Java 21 and SQLite, fix the seed-data restart bug, and verify the build and tests.